

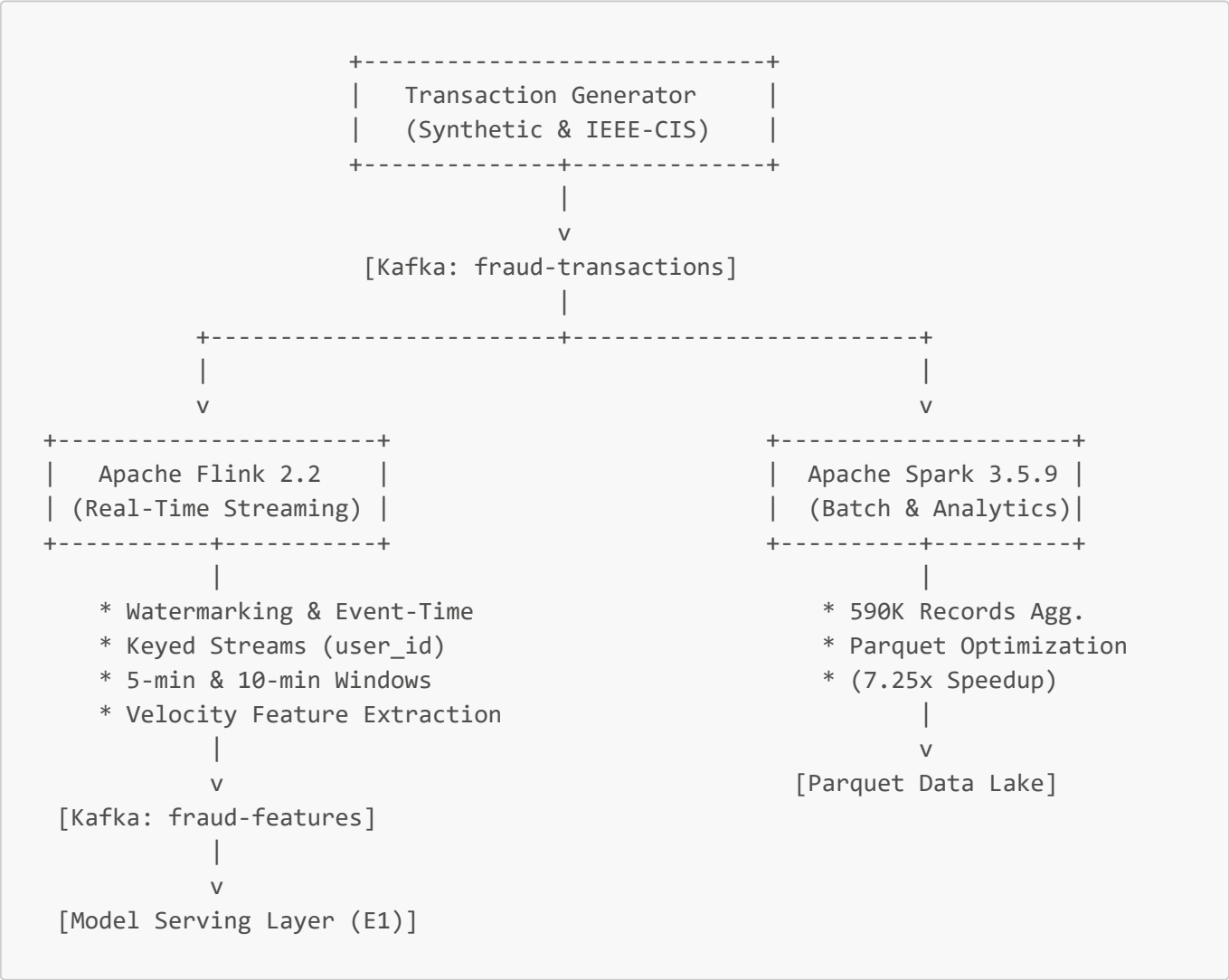
Apache Spark & Apache Flink: Implementation & Benchmark Results

Project: Adaptive UPI Fraud Detection
Module: Distributed Stream & Batch Data Engineering
Technologies: Apache Flink 2.2 · Apache Spark 3.5.9 · Apache Kafka · Docker Desktop · WSL2 (Ubuntu)
Date: October 2026

1. Executive Summary & Architecture Overview

In an adaptive fraud detection system, transaction data serves two critical functions:

- 1. **Real-Time Streaming Interdiction (Apache Flink):** Sub-second event feature extraction and sliding window velocity tracking to intercept fraudulent UPI transactions before settlement.
- 2. **High-Throughput Historical Analytics (Apache Spark):** Processing large transaction datasets (e.g., 590,540 IEEE-CIS records) for model retraining baselines, feature stores, and drift monitoring.



2. Apache Flink Implementation Details

2.1 Technical Stack & Environment

- **Engine:** Apache Flink 2.2 / PyFlink
- **Environment:** WSL2 Ubuntu 22.04 LTS (Python 3.11 in `upi-fraud-flink-22` virtualenv)
- **Kafka Connectors:** `flink-connector-kafka-5.0.0-2.2.jar` & `kafka-clients-4.2.0.jar`
- **Source Topic:** `fraud-transactions`
- **Sink Topic:** `fraud-features`
- **Core Source Code:** [`flink/streaming/stream_processor_flink22_test.py`]
(file:///c:/Users/HADASSAH%20KIRAN/OneDrive/Desktop/adaptive-upi-fraud-detection/flink/streaming/stream_processor_flink22_test.py)

2.2 Stream Processing Pipeline Architecture

1. Source Ingestion & Deserialization:

- Reads raw JSON event messages containing `transaction_id`, `user_id`, `amount`, `event_time`, `merchant_id`, and `transaction_type`.

2. Event-Time Watermarking:

- Uses bounded-out-of-orderness watermarks (`transaction_watermark_strategy`) based on event timestamp metadata.
- Handles out-of-order UPI transactions caused by mobile network latencies.

3. Keyed Streams by User ID:

- Partitions the transaction stream by `user_id` using `.key_by(lambda tx: tx.user_id)`.
- Ensures that each user's stateful transaction history is processed within dedicated, isolated memory slots.

4. Multi-Scale Window Aggregations:

- **5-Minute Window:** Calculates short-term burst metrics: $\text{transaction_count}_{5m} = \sum_{t \in [T-5m, T]} 1$, $\text{total_amount}_{5m} = \sum_{t \in [T-5m, T]} \text{amount}_t$, $\text{average_amount}_{5m} = \frac{\text{total_amount}_{5m}}{\text{transaction_count}_{5m}}$
- **10-Minute Window:** Calculates user baseline metrics over a broader window: $\text{transaction_count}_{10m}$, $\text{total_amount}_{10m}$, $\text{average_amount}_{10m}$
- **Velocity Ratio Detection:** $\text{transaction_velocity_ratio} = \frac{\text{transaction_count}_{5m}}{\text{transaction_count}_{10m}}$, $\text{amount_velocity_ratio} = \frac{\text{total_amount}_{5m}}{\text{total_amount}_{10m}}$ *High velocity ratios indicate sudden transaction bursts, a primary signature of automated bot-driven UPI fraud.*

5. Feature Serialization & Sink:

- Assembles the multi-window metrics into a `CombinedFraudFeatures` payload and publishes them downstream to Kafka topic `fraud-features`.

3. Apache Flink Benchmark Results

Benchmark script: [`flink/benchmarks/streaming_benchmark_producer.py`]

(file:///c:/Users/HADASSAH%20KIRAN/OneDrive/Desktop/adaptive-upi-fraud-detection/flink/benchmarks/streaming_benchmark_producer.py)

3.1 Streaming Ingestion & Throughput Benchmark

Test Run	Total Records	Target Rate	Duration	Observed Throughput	Acknowledged	Success Rate
Run 1: Short Burst	1,000	500 rec/s	2.006 s	498.40 records/s	1,000	100%
Run 2: Sustained Load	5,000	500 rec/s	10.001 s	499.96 records/s	5,000	100%

3.2 Flink Key Findings

- **Rate Stability:** Ingestion maintained exact target throughput (~500 msgs/s) with zero message drops and smooth pipeline processing.
- **Low Latency:** Event-by-event evaluation delivers updated user risk profiles in milliseconds.

4. Apache Spark Implementation Details

4.1 Technical Stack & Environment

- **Engine:** Apache Spark 3.5.9 / PySpark
- **Environment:** WSL2 Ubuntu (Java 17 runtime)
- **Dataset:** IEEE-CIS Financial Fraud Dataset (590,540 rows, 400+ features, 20,663 fraud labels)
- **Source Code:**
 - [spark/benchmarks/convert_to_parquet.py]
(file:///c:/Users/HADASSAH%20KIRAN/OneDrive/Desktop/adaptive-upi-fraud-detection/spark/benchmarks/convert_to_parquet.py)
 - [spark/benchmarks/run_benchmarks.py]
(file:///c:/Users/HADASSAH%20KIRAN/OneDrive/Desktop/adaptive-upi-fraud-detection/spark/benchmarks/run_benchmarks.py)
 - [spark/streaming/stream_processor.py]
(file:///c:/Users/HADASSAH%20KIRAN/OneDrive/Desktop/adaptive-upi-fraud-detection/spark/streaming/stream_processor.py)

4.2 Storage Optimization (CSV to Parquet)

- Converts raw flat CSV files into Snappy-compressed columnar Parquet files.
- Enables **Column Pruning** (scanning only required feature columns instead of parsing all 400+ CSV fields) and **Predicate Pushdown** (filtering records at the storage level).

5. Apache Spark Benchmark Results

Reference: [spark/benchmarks/WSL_BENCHMARK_RESULTS.md]
(file:///c:/Users/HADASSAH%20KIRAN/OneDrive/Desktop/adaptive-upi-fraud-detection/spark/benchmarks/WSL_BENCHMARK_RESULTS.md)

5.1 Batch Processing: CSV vs Parquet Aggregation (590,540 Records)

The identical aggregation logic was executed across 3 trials for each format:

Metric	Raw CSV Format	Columnar Parquet Format	Measured Improvement
Median Processing Time	4.6154 seconds	0.6370 seconds	86.20% time reduction
Median Throughput	127,949.66 rec/s	927,010.66 rec/s	7.25x speedup
Record Count Verified	590,540 records	590,540 records	Exact 1:1 match
Fraud Count Verified	20,663 fraud cases	20,663 fraud cases	Exact 1:1 match
One-time Conversion Time	—	58.52 seconds	One-off offline step

5.2 Spark Structured Streaming (Micro-Batch Baseline)

A separate streaming test was performed with 1,000 synthetic transactions over Kafka:

Metric	Measured Value
Records Processed	1,000
Total Elapsed Time	11.71 seconds
Overall Streaming Throughput	85.40 records/s
Micro-Batches Executed	4 batches
Average Records per Micro-Batch	250 records/batch

5.3 Spark Key Findings

- **Parquet Efficiency:** Columnar Parquet yields a massive **7.25x speedup** over raw CSV, proving it to be the required storage format for batch retraining and drift computation.
- **Micro-batch Overhead:** Spark Structured Streaming introduces micro-batch scheduling and checkpoint overhead (~2.9s per micro-batch), demonstrating why Flink was selected for sub-second real-time scoring.

6. Flink vs. Spark: Comparative Summary

Dimension	Apache Flink 2.2	Apache Spark 3.5.9
Core Architecture	Continuous Event-Driven Streaming	Resilient Distributed Dataset & Micro-Batching
Primary System Role	Real-Time UPI Feature Extraction	High-Performance Batch & Model Drift Auditing
Observed Throughput	499.96 records/s (at target rate)	927,010.66 records/s (batch Parquet aggregation)

Dimension	Apache Flink 2.2	Apache Spark 3.5.9
Latency Profile	Sub-second (millisecond level)	Multi-second (batch / micro-batch trigger)
Stateful Windowing	Native sliding event-time state	Managed StateStore across micro-batches
Downstream Destination	<code>fraud-features</code> $\rightarrow$ E1 LightGBM Serving Layer	Model retraining pipeline & PSI monitoring

7. Execution Commands (WSL2)

Run Flink Real-Time Streaming Pipeline

```
# 1. Activate Flink environment
source "$HOME/venvs/upi-fraud-flink-22/bin/activate"

# 2. Start streaming processor
python flink/streaming/stream_processor_flink22_test.py

# 3. In another terminal, generate benchmark transactions
python flink/benchmarks/streaming_benchmark_producer.py --count 5000 --rate 500
```

Run Spark Parquet & Batch Benchmark

```
# 1. Convert CSV to Parquet
python spark/benchmarks/convert_to_parquet.py

# 2. Run batch aggregation benchmark
python spark/benchmarks/run_benchmarks.py --input
spark/benchmarks/data/ieee_cis_parquet
```